

COMP 512 – Programming Assignment 1

Distributing a Travel Reservation System

Group 42

Arman Ahmed (McGill ID: 261147631)

Frédéric Lahaie-Bertrand

October 5, 2026

1 Overview and Shared Architecture

The goal of this assignment was to distribute a travel reservation system implemented in Java. We were initially provided with a code template where a single client was talking over RMI to a single ResourceManager (RM) that stores all flights, cars, rooms, and customers in a single hashmap. The system was first distributed with RMI, by inserting a middleware between the (unmodified) clients and three RMs, one for each resource type (Section 2). The system was then distributed using TCP sockets (Section 3).

Both versions share a similar architecture. The client is unchanged and talks only to a middleware server. The middleware dispatches every request to one of the three RMs, one each for flights, cars, and rooms. Each RM runs the ResourceManager class provided in the starter code (which implements the whole IResourceManager interface), and the middleware only calls flight methods on the flights RM, car methods on the cars RM, and room methods on the rooms RM. The RMI version uses the ResourceManager class unchanged, and the TCP version adds a cancelReservation operation (used to undo a bundle) and a total line to the customer's bill.

1.1 Customers

In both versions, we chose to replicate customers on all three RMs so that each RM holds a record for every customer, but each copy only contains the reservations of that RM's resource type. This simplifies the code since every reservation is a single call to one RM and we can reuse the template's reserveItem: the function finds the customer, decrements the resource count, and records the reservation in the customer's data, all in the same RM. A reservation thus can't half-succeed, and this approach requires no change to the RM code. The drawback is that creating, deleting, and querying a customer involves all three RMs. This means that a creation or deletion that fails after only one or two of the RMs were updated leaves the copies inconsistent and must be handled somehow. The total bill also needs to be assembled from the 3 RMs.

The specification also offers two alternatives to this design: store the customers in an additional server or at the middleware, i.e. store all the customers at a single place. This simplifies creating customers, queries about their reservations, and computing the total bill (deleting a customer still involves every RM that holds their reservations), but it separates the customers from the items they reserve (every reservation must update two servers). While both designs may result in operations that can fail halfway, and thus require an inconsistency to be repaired, they differ when a server crashes: a crashed RM in our design only loses its own items together with their reservations, whereas if the middleware or the customer server fails, all customer data is lost.

1.2 Bundles

A bundle reserves one or more flights, and possibly a car and/or a room at a location, for a single customer. Reserving a bundle is handled by the middleware in both versions since it can involve up to three RMs (one per resource type). A bundle must be all-or-nothing, i.e. either every requested item is reserved, or none is. The difficulty in achieving that is that the template offers no way to cancel a single reservation (deleting a customer would also cancel their previous reservation). The two versions handle this differently (see Sections 2 and 3).

2 RMI Distribution

2.1 Middleware and startup

The middleware is implemented using only one class (`Server.RMI.RMIMiddleware`) that implements `IResourceManager`. Since the client interacts only with that interface, it connects to the middleware the same way it would to an RM using `./run_client.sh <host> Middleware`. The middleware receives the hostnames of the flights, cars, and rooms RMs as three command line arguments. On startup, it then looks up each of the three RMs in the registry, retrying every 500ms until they all become reachable. It only binds itself as `group_42.Middleware` after all the RMs are connected so that a client can't reach the middleware if it's not fully initialized. This also means that the start order of the processes don't matter. The interface methods are just forward methods to the corresponding RM.

2.2 Errors and exceptions

When an RM is down or does not answer within 5 seconds, the middleware's call to that RM throws a `RemoteException`. We chose not to catch this exception in the middleware: RMI delivers it to the client wrapped in a `ServerException`, and the client already reports it by printing it as a command error with its cause. If the middleware itself is down, the client automatically reconnects to it and executes the command again.

2.3 Customers

As described above, customers are replicated in each RM and so the middleware sends every customer creation, deletion, and bill query to all three RMs. To create a customer, `AddCustomer` chooses IDs sequentially (via a counter kept in the middleware), skipping any ID that is already taken (e.g. by `AddCustomerID`), and creates the customer in each RM. If this fails for one RMs, the customer is deleted from the RMs where the creation succeeded (safe since the customer has no reservations yet) and the exception is propagated to the client. Deleting a customer can't be undone (the interface has no way to restore the reservations of the deleted customer), so the middleware calls `deleteCustomer` on all RMs and throws an exception to the client if one fails. The client then must run the command again to delete the remaining customer (or else it'll be leaked along with its items).

2.4 Bundle

As discussed previously, the bundles must be all-or-nothing, but the provided interface has no way of canceling a single reservation without wiping a customer's previous reservations. The RMI version handles this by checking that all the resources are available before reserving anything. This is done in three phases (using the middleware's global lock):

1. Check that each argument is valid.
2. Pre-check that the customer exists on every RM involved in the bundle and that the resources are available. A failure at this stage returns `false` without doing anything.
3. Reserve the flights, and possibly the car and the room.

The lock prevents other clients from modifying anything between the second and third step, and so this operation never fails unless a RM fails during the third part. If this happens, the middleware throws an exception to the client that lists reservations that were made and the user must complete the bundle manually.

2.5 Concurrency

The middleware handles concurrent requests by using one global lock held by every operation that may modify any state in a RM (the only operations that are not locked are single item queries). This means that most operations are fully serialized but it makes the implementation simpler. Because a RM could hang (not crash) while holding the lock, we added a timeout (5 seconds) so that these calls fail instead of blocking every client forever.

2.6 Final version

Our final version is the version developed with AI, which is the one described above. The version written by hand had several issues involving concurrency, the support for multiple RMs per resource type (which is a more general solution but introduced several bugs), and other behaviour that didn't match the specification. Modifying this version would've amounted to a rewrite, only to end up with a version that essentially mirrors the one developed with AI. Since the AI version was relatively well tested, we chose to submit it as our final version.

3 TCP Sockets

3.1 Message Passing

Our implementation employs a persistent socket for communication between the middleware and each client, and uses a fresh socket for communication between the middleware and the resource managers. This implementation has its benefits in its simplicity of ensuring correctness. Since the clients send blocking requests, we can ensure that any message the middleware is sending any given client is for the last request sent. This cannot be ensured between the middleware and resource managers. We can illustrate this problem most easily by way of the example below.

Suppose, instead, that we had persistent sockets to each resource manager and two clients send requests for a flight operation, say querying two different flights at around the same time. Since the resource managers and middleware are concurrent, we cannot assume that one of said requests will be answered before the other. Further, in message reception, we will have no way to distinguish the result of one request from the other, given they are the same type.

Solving this would require implementing some sort of message tagging system, for example sequence numbers, to disambiguate messages from another. Further, we would have had to implement a mechanism to route messages to the right requests. One possible avenue to solve this is to have the resource managers repeatedly send results until they are received by the correct request, though this could seriously hinder the latency and jitter of the infrastructure.

The creation (and destruction) of a new socket for each request, then, avoids these issues. Issues of disambiguation are solved because each socket is used only for the middleware to send a request to the RM, and for the RM to receive it, keeping the code simple and correctness easily established. This method, though, will see increased overhead for each request because of the repeated creation and destruction of the sockets.

Messages are passed using a Request object consisting of a String for a command and an Object array for the arguments. While this approach makes the parsing of each object easy, we lose the protection of the type-checker here. The use of an Object to send messages also allows us to rely on Java's serialization during sending and reception. A similar response class is used for the reception of results from the resource managers.

3.2 Concurrency

Concurrency is handled simply. Upon connection, the middleware assigns each client its own thread which will handle of its receives and sends. Since the clients are blocking, this allows the middleware to be maximally concurrent, dealing with a request from each client concurrently. Resource managers create a new thread upon connection of a new socket from the middleware. These are managed by a state lock at each resource manager when the operation executes. Queries take the read-lock, so multiple of them can run together, where mutators take a write lock and are therefore treated as one operation. This allows for a good amount of speed gain due to concurrency, allowing for many reads to occur at once, but allows for the easy assurance of correctness. Further, this approach does not guarantee a FIFO order across different clients; operations occur within a single RM based on how quickly they can take the lock.

3.3 Customers and Bundles

Customer data is replicated at all three resource managers, with the middleware coordinating their creation deletion, and bill creation. Bundles are all-or-nothing; if one of its reservations fails it rolls back all of the reservations in the bundle. If one of these cancellations fails, the middleware reports an error.

4 Testing

Listed below are some of the (more interesting) tests used to determine the correctness of our implementations. Some tests are not listed explicitly so that the list fits in one page, but a short description is provided instead. These tests were conducted locally and on the machines in Trottier.

RMI

- *Dispatch*. Resources are only stored on the RM of their type (checked by querying RMs directly).
- *Delete item*. Deleting an item only works once, deleting it again returns false.
- *Delete customer*. The count of all seat, car, and room are restored on all RM when deleting a customer holding resources.
- *Negative counts*. Command like `AddFlight,2,-5,10` returns false and the RM is never reached.
- *Price update*. `AddFlight,1,0,250` on an existing flight only changes its price (since the number of seats to add is 0). A customer who reserved a seat before a price update will only be charged the previous price, but if he reserves a new seat afterwards, he will be charged the new price for all his seats. Total bill is correctly calculated.
- *Bundle*. If a room is sold out, the bundle fails and the other resource counts remain unchanged. Bundles with non-existing customer or resource fail.
- *Customer IDs*. `AddCustomer` returns 1, `AddCustomer` again returns 2. After `AddCustomerID,3` and `AddCustomerID,5`, running it twice returns 4, then 6. A duplicate ID returns false.
- *RM failure*. Commands involving a killed RM fail with an exception.

TCP

These tests were focused primarily on concurrency, some of them include:

- Concurrent reservations
- Concurrent customer creation
- Concurrent reserves and deletes
- Rollback of a Bundle

5 Use of AI and Token Estimate

The RMI version with AI was developed by Frédéric using Claude (Sonnet 5.5), while Arman developed the version without AI. The TCP version was developed by Arman using Copilot (Copilot CLI agent in VS Code in auto mode, using GPT-6 Luna). In both cases, we made the architecture decisions. The AI would present us with options and their trade-offs, and would implement only what had been agreed upon, step by step. Each step was tested independently to make it easier for us to understand the resulting code. The report and the slides were written without the use of AI.

The Copilot conversation is attached as a JSON file, while the Claude conversation is accessible via this link (we couldn't export the conversation as a PDF).

5.1 Tokens

For the RMI session, Claude estimates around 8-10 million input tokens and 140k output tokens (for the conversation itself). For the TCP version, Copilot reports about 9.8 million input tokens and 97k output tokens. In both cases, most input tokens are the conversation being re-read on every request. In total, we used roughly 18-20 million input tokens and 240k output tokens.

5.2 Some thoughts

Both of us agree that the use of AI, while allowing us to finish the assignment faster, greatly impaired our ability to reason about the code, even though we “made the decisions”. It also left us with a relatively shallow understanding of the concepts involved.

Overall, we would rather not use AI for future assignments, and postpone our inevitable transition into professional meat proxies for LLMs.

6 Collaboration

Meetings

- (September 29, 30 minutes) Go through the template code, discuss what needed to be done, and who does the AI implementation.
- (October 2, 1 hour) Compare and test our implementations, decide on which implementation to use, and distribute the rest of the work among us.

Outside of meetings, we have been continuously communicating via Discord every day throughout the project.

Contributions

- Arman: RMI version by hand, TCP version using AI, writing the report (TCP section), preparing the demo
- Frédéric: RMI version using AI, testing both RMI and TCP versions, writing the report (all sections, except for some parts of TCP), preparing the demo